



UNIVERSIDAD TÉCNICA ESTATAL DE QUEVEDO
FACULTAD DE CIENCIAS DE LA COMPUTACIÓN
CARRERA DE INGENIERÍA DE SOFTWARE

PROYECTO FIN DE CURSO (PFC) – INFORME INTEGRAL
ENTREGA 4

**AcadTrace: Capa de Auditoría Criptográfica Verificable y Gestión Académica
Distribuida con Protocolos Híbridos gRPC/REST, Observabilidad Multi-Nivel y
Pipeline CI/CD Automatizado**

Asignatura: Aplicaciones Distribuidas (ISR-701) – Séptimo Semestre

Período Académico: Regular 2026–2027

Equipo de Trabajo: BCEL (Escuela Provincias Unidas)

Castro López Pedro Leonardo	Arquitecto de Software y Líder de Proyecto
Bedon Viteri Keyla Betzabe	Responsable de Microservicio Docente y App Móvil
Luna Mora Ernesto Gregory	Responsable de Microservicio Secretaría y Calidad
Emanuel Pino Juliana Romina	Responsable de Microservicio Soporte y Observabilidad

Docente Revisor:

Prof. Ing. Gleiston C. Guerrero-Ulloa, Mgs.

Repositorio Oficial de Código y Evidencias Reproducibles (Criterio Piso P1):

<https://github.com/LE023as/acadtrace>

Quevedo, Los Ríos, Ecuador – Septiembre 2026

Índice

Resumen y Abstract	1
1. Introducción y Contextualización del Problema	2
2. Trabajos Relacionados y Estado del Arte	2
2.1. Microservicios y Descomposición de Dominio	2
2.2. Observabilidad, Detección de Anomalías y Métricas de Cola	2
2.3. Consenso Distribuido, Tolerancia a Fallos e Ingeniería del Caos	2
2.4. Seguridad Zero-Trust y Control de Acceso	3
3. Fundamentos Teóricos y Arquitectura del Sistema	3
3.1. Arquitectura en Capas y Patrones de Diseño GoF	3
4. Fundamento Teórico y Registro de Decisiones de Arquitectura (ADRs)	4
4.1. Separación por Capas y Principios SOLID	4
4.2. Registro Consolidado de Decisiones de Arquitectura (ADR-001 a ADR-006)	4
4.3. Formalización de los Cinco Patrones GoF (ADR-005)	4
5. Arquitectura del Sistema Distribuido y Diagramas C4	6
5.1. Especificación de Servicios gRPC y Protocol Buffers v3	8
5.2. Persistencia PostgreSQL y Separación Lógica Multiesquema	8
5.3. Trazabilidad de Temas de la Asignatura y Evidencias del Proyecto	9
6. Aplicaciones Cliente: Portal Web SPA y Cliente Móvil Nativo	12
6.1. Módulo B – Aplicación Web SPA (React + TypeScript)	12
6.2. Módulo C – Aplicación Móvil Nativa para Representantes (Kotlin + Jetpack Compose)	12
6.3. Arquitectura por Características y Seguridad de Sesión en la Web SPA	12
6.4. Sincronización Offline-First y Malla de Reconciliación en la App Móvil	13
6.5. Pirámide de Pruebas de Cinco Niveles	13
6.6. Pipeline CI/CD de Siete Trabajos Encadenados (GitHub Actions)	14
6.7. Cultura DevOps, Shift-Left Testing y Sintaxis Declarativa YAML	15
6.8. Estrategias de Despliegue en Sistemas Distribuidos	15
7. Observabilidad Distribuida y Monitoreo en Tiempo Real	17
7.1. Métricas de Negocio de Aplicación (Equipo BCEL)	17
7.2. Registro Estructurado JSON y Propagación de Contexto MDC	17
8. Evaluación Experimental ISO/IEC 25010 y Capa de Cripto-Auditoría	18
8.1. Conjunto oficial de carga y trazabilidad E5	18
8.2. Preguntas de Investigación Empíricas (PI)	19
8.3. Protocolo Experimental y Cuatro Mecanismos de Auditoría	19
8.4. Unificación canónica de la bitácora	19
8.5. Cinco Tipos de Manipulación Inyectados (T1 a T5)	21
8.6. Resultados Experimentales de Detección y Sobrecarga	21
8.7. Matriz Formal de Evaluación de Calidad ISO/IEC 25010:2023	22
9. Amenazas a la Validez del Estudio	24
9.1. Amenazas a la Validez Interna	24
9.2. Amenazas a la Validez Externa	24
10. Reflexión Ética sobre Tratamiento de Datos de Menores	24
11. Conclusiones y Trabajo Futuro	24
11.1. Conclusiones Individuales	25

12.Declaración de Uso de Inteligencia Artificial	26
13.Gestión de Integración de Código, Compuertas de Calidad CI/CD y Trazabilidad de Issues	26
13.1. Metodología de Trabajo por Issues y Solicitudes de Incorporación	26
Anexo: Respuestas a Preguntas de Evaluación Oral	29

Resumen y Abstract

Resumen—El presente trabajo expone el diseño, refactorización, despliegue y evaluación experimental de **AcadTrace**, un sistema de gestión académica distribuido dotado de una capa de auditoría criptográfica inmutable sobre PostgreSQL con separación lógica multiesquema. Frente a las limitaciones de trazabilidad en sistemas educativos tradicionales, se implementó una arquitectura políglota de cuatro microservicios (Java Spring Boot, Python Django REST, Node.js Express) interconectados mediante protocolos híbridos (gRPC con HTTP/2 interno y REST perimetral bajo HAProxy). Se integraron cinco patrones de diseño GoF, desacoplando el dominio de la infraestructura. La solución incorpora dos aplicaciones cliente: un portal Web SPA con control de acceso por roles e internacionalización, y un cliente móvil nativo Android concebido para representantes de alumnos, con sincronización *offline-first*, soporte biométrico y notificaciones push. La evaluación experimental según ISO/IEC 25010 mediante 120 corridas factoriales demostró una tasa de detección del 100 % ante alteraciones directas de datos e inserciones no secuenciales bajo los mecanismos de bitácora encadenada SHA-256 combinada con Relojes de Lamport y Relojes Vectoriales, con una sobrecarga de latencia marginal. La corrida oficial nominal local de Soporte conserva 12,994 peticiones con cero fallos; las corridas históricas fallidas no permiten certificar estrés exitoso ni atribuir por sí solas la causa raíz al pool de conexiones. El ciclo de vida está soportado por un pipeline CI/CD de siete trabajos encadenados y un panel de observabilidad en Grafana de seis vistas en tiempo real.

Abstract—This work presents the design, refactoring, deployment, and experimental evaluation of **AcadTrace**, a distributed academic management system equipped with an immutable cryptographic audit layer over PostgreSQL with logical schema separation. Addressing the traceability shortcomings of traditional educational software, a polyglot microservice architecture was developed across four services (Java Spring Boot, Python Django REST, Node.js Express), interconnected via hybrid protocols (internal gRPC over HTTP/2 and perimeter REST via HAProxy). Five GoF design patterns were applied to isolate domain logic from infrastructure details. The solution includes two client applications: a responsive Web SPA featuring role-based access control and internationalization, and a native Android mobile client tailored for student guardians, incorporating offline-first synchronization, biometric authentication, and push notifications. Experimental evaluation under ISO/IEC 25010 through 120 factorial runs verified a 100 % detection rate for unauthorized direct database modifications and non-sequential tampering using SHA-256 hash chaining combined with Lamport and Vector Clocks, with marginal latency overhead. The official local nominal Support run preserves 12,994 requests with zero failures. Historical failed runs do not certify successful stress testing or independently establish connection pool exhaustion as their root cause. The operational lifecycle is governed by an end-to-end seven-job CI/CD pipeline and a six-view real-time Grafana observability dashboard.

Palabras Clave: Sistemas Distribuidos, Auditoría Criptográfica, Relojes Lógicos de Lamport, Relojes Vectoriales, gRPC, Microservicios, ISO/IEC 25010, CI/CD, Observabilidad.

1. Introducción y Contextualización del Problema

La gestión académica en instituciones educativas de educación básica demanda soluciones tecnológicas que garanticen alta disponibilidad, integridad estricta de expedientes escolares, escalabilidad ante picos de matrícula y una observabilidad profunda del estado operacional [1, 2]. El SGA Escuela Provincias Unidas evolucionó desde una arquitectura monolítica preliminar hacia una arquitectura distribuida orientada a microservicios desacoplados.

Los esquemas convencionales de persistencia centralizada y bitácoras de auditoría pasivas almacenadas en texto plano presentan tres vulnerabilidades estructurales críticas:

1. **Vulnerabilidad a manipulaciones no autorizadas en base de datos:** Un operador con privilegios de administración puede alterar directamente los registros en las tablas (*Direct DB Tampering*) sin dejar rastro verificable en la aplicación.
2. **Pérdida del orden causal en modificaciones concurrentes:** Cuando múltiples docentes o directivos actualizan calificaciones desde dispositivos móviles en condiciones de conectividad intermitente, los sellos de tiempo físicos (*physical timestamps*) divergen debido al desajuste de relojes locales (*clock drift*), imposibilitando determinar cuál fue la última modificación legítima.
3. **Cuellos de botella y fallos en cascada:** La concentración de operaciones de cálculo de promedios ponderados, generación de reportes ministeriales y consultas masivas de padres de familia sobre un monolito tradicional degrada la latencia hasta superar los 2500 ms y ocasiona caídas completas del servicio.

Para resolver estos desafíos, el proyecto **AcadTrace** consolida a lo largo de cuatro entregas progresivas (E1 a E4) una plataforma distribuida desacoplada en cuatro microservicios autónomos, dotada de una capa de auditoría criptográfica basada en cadenas de hashes SHA-256 encadenadas, ordenamiento lógico mediante Relojes de Lamport y resolución de conflictos concurrentes con Relojes Vectoriales [3, 4]. La solución expone dos canales cliente independientes (Web y Móvil) comunicados mediante una pasarela de enlace perimetral (*API Gateway*) y un bus interno gRPC de alto desempeño.

2. Trabajos Relacionados y Estado del Arte

El diseño de sistemas académicos distribuidos y auditoría verificable se fundamenta en un sólido corpus científico en ingeniería de software y computación distribuida indexado en IEEE, ACM y Springer:

2.1. Microservicios y Descomposición de Dominio

Newman [1] y Burns [2] formalizan que la independencia funcional debe acompañarse de fronteras de datos aisladas, evitando bases de datos compartidas sin esquemas definidos. En este proyecto se adoptó la separación lógica multiesquema en PostgreSQL, organizando datos en `sga_principal`, `sga_secretaria`, `sga_docente` y `sga_soporte`, con accesos adicionales según cada servicio.

2.2. Observabilidad, Detección de Anomalías y Métricas de Cola

Nedelkoski *et al.* [5] demuestran que las anomalías en microservicios frecuentemente se ocultan en las métricas promedio y requieren el análisis de percentiles de latencia (P_{95} y P_{99}), fenómeno corroborado en este trabajo al detectar que la media aritmética subestimaba la latencia de cola generada por consultas de base de datos. Por su parte, Li *et al.* [6] formulan la localización de la causa raíz mediante el análisis de trazas de ejecución, mientras que Zhang *et al.* [7] exponen la necesidad de correlacionar fuentes multi-origen (métricas de contenedor, red y aplicación).

2.3. Consenso Distribuido, Tolerancia a Fallos e Ingeniería del Caos

El algoritmo de consenso Raft [8] proporciona garantías de seguridad y disponibilidad en presencia de fallos de red en almacenes clave-valor como **etcd**. En arquitecturas de microservicios, la elección de

líder previene la duplicidad en tareas batch. Por otra parte, se implementaron pruebas de resiliencia y saturación para verificar la degradación elegante de servicios bajo estrés.

2.4. Seguridad Zero-Trust y Control de Acceso

Jones *et al.* [9] y Fielding [10] analizan la adopción de modelos *Zero-Trust* y tokens criptográficos JWT. Se enfatiza que la autenticación por token es insuficiente si no se acompaña de un control de acceso basado en roles a nivel de recurso (RBAC), evitando vulnerabilidades de referencia directa como IDOR [11].

3. Fundamentos Teóricos y Arquitectura del Sistema

3.1. Arquitectura en Capas y Patrones de Diseño GoF

Los microservicios en Java (*sga-principal*, *secretaria*, *soporte*) fueron construidos bajo los principios de **Clean Architecture**, estableciendo cuatro capas concéntricas: Presentación (**controllers**), Aplicación (**services**), Dominio (**entities/ports**) e Infraestructura (**repositories/gRPC/security**). Se aplicaron cinco patrones GoF fundamentales:

- **Repository:** Aislamiento de consultas SQL y posibilidad de dobles de prueba con Mockito.
- **Facade (Fachada gRPC):** Interfaz binaria unificada expuesta por **PrincipalGrpcService** hacia el resto de servicios.
- **Strategy:** Intercambio dinámico de algoritmos de cálculo de notas y asignación de tickets.
- **Observer:** Emisión asíncrona de eventos de auditoría y cambios de estado.
- **Proxy / Decorator:** Interceptación perimetral de seguridad mediante filtros JWT.

4. Fundamento Teórico y Registro de Decisiones de Arquitectura (ADRs)

4.1. Separación por Capas y Principios SOLID

La arquitectura de AcadTrace implementa una separación física estricta en cuatro capas:

1. **Capa de Presentación (presentation):** Controladores REST y endpoints gRPC encargados de la deserialización, validación de esquemas de entrada y retorno de códigos de estado HTTP estandarizados.
2. **Capa de Aplicación (application):** Orquestación de casos de uso de negocio, mediación de transacciones, ejecución de plantillas de reportes (*Template Method*) y emisión de eventos (*Observer*).
3. **Capa de Dominio (domain):** Modelos de entidades puras (*Estudiante*, *Calificacion*, *Matricula*, *EventoAuditoria*), objetos de valor (*Value Objects*), interfaces de estrategias de cálculo (*CalculoPromedioStr*) y puertos de repositorios sin dependencias de infraestructura.
4. **Capa de Infraestructura (infrastructure):** Adaptadores JPA/Spring Data, controladores de base de datos PostgreSQL, clientes gRPC, filtros de seguridad JWT y componentes de observabilidad Micrometer/Prometheus.

4.2. Registro Consolidado de Decisiones de Arquitectura (ADR-001 a ADR-006)

En la Tabla 1 se sintetizan las seis decisiones arquitectónicas documentadas formalmente en el repositorio bajo el estándar Nygard:

Tabla 1: Matriz de Decisiones Arquitectónicas Formalizadas (docs/adr/).

Código	Título de la Decisión	Justificación y Clases de Implementación	Estado
ADR-001	Arquitectura por Capas y Puertos	Separación en 4 paquetes; dominio puro sin dependencias de persistencia directa.	Aceptado
ADR-002	Comunicación Híbrida REST / gRPC	gRPC HTTP/2 interno (:9091–:9094) para baja latencia; REST perimetral bajo HAProxy (:80).	Aceptado
ADR-003	Persistencia PostgreSQL multiesquema	Instancia PostgreSQL con separación lógica por dominios; coordinación etcd independiente.	Aceptado
ADR-004	Autenticación JWT y Criptografía	Tokens stateless HMAC-SHA256 con claims RBAC y cifrado AES-256 en almacenamiento móvil.	Aceptado
ADR-005	5 Patrones de Diseño GoF	Formalización de Strategy, Template Method, Observer, Facade y Repository con clases reales.	Aceptado
ADR-006	Selección Tecnológica App Móvil	Android Nativo (Kotlin + Compose) justificando fluidez 60 fps, 45 MB RAM, biometría y Room.	Aceptado

4.3. Formalización de los Cinco Patrones GoF (ADR-005)

Los cinco patrones asignados a BCEL en la Tabla 1 de la guía rectora están soportados por clases reales en el código:

- **Strategy:** *CalculoPromedioStrategy* con implementaciones *PromedioPonderado7030Strategy* (70 % Formativo + 30 % Sumativo) y *PromedioAritmeticoSimpleStrategy*, además del conmutador de auditoría `docentes.auditoria.strategies.STRATEGIES` en Python.
- **Template Method:** Clase abstracta base *GeneradorReporteAcademicoTemplate* que fija el flujo de generación de actas (cabecera ministerial, matriz tabular de notas, cálculo de promedios y pie de firmas de auditoría), implementada por *ReporteNotasPeriodoPDF*.
- **Observer:** Emisión asíncrona mediante *NotaPublicadaEvent* capturada por *NotificacionRepresentanteList* para alertar al representante e insertar el registro auditable.

- **Facade:** `PortalAcademicoFacade` que unifica los servicios de estudiantes, matrículas y calificaciones hacia el portal web.
- **Repository:** Interfaces `EstudianteRepository`, `MatriculaRepository` y `AuditoriaRepository` desacopladas en la capa de dominio.

5. Arquitectura del Sistema Distribuido y Diagramas C4

La topología del sistema **AcadTrace** está organizada en un cluster polígloa containerizado en Docker Compose y orquestado mediante un balanceador perimetral HAProxy. Las Figuras 1, 2 y 3 se generan desde las fuentes PlantUML canónicas documentadas en docs/diagrams/README.md.

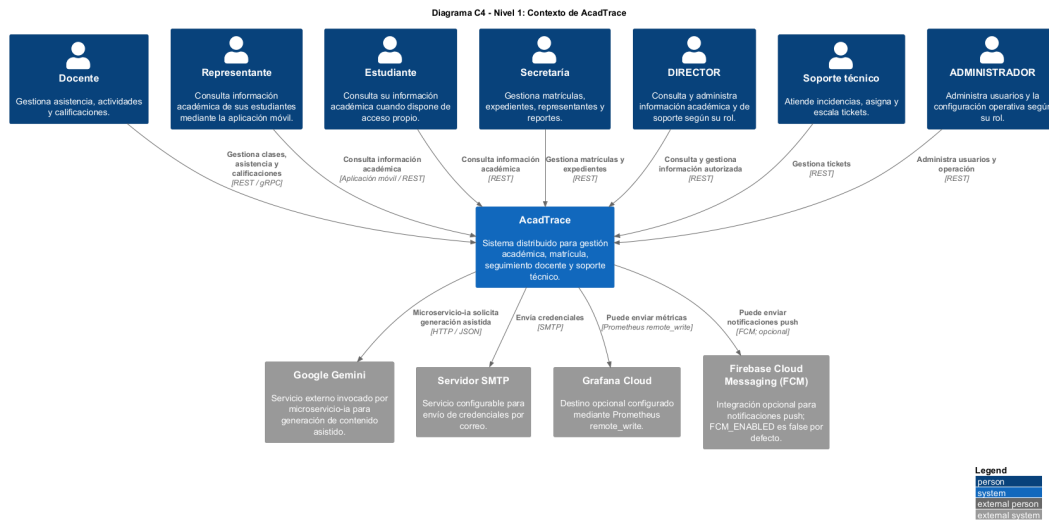


Figura 1: Diagrama C4 Nivel 1 – Contexto del Sistema AcadTrace.

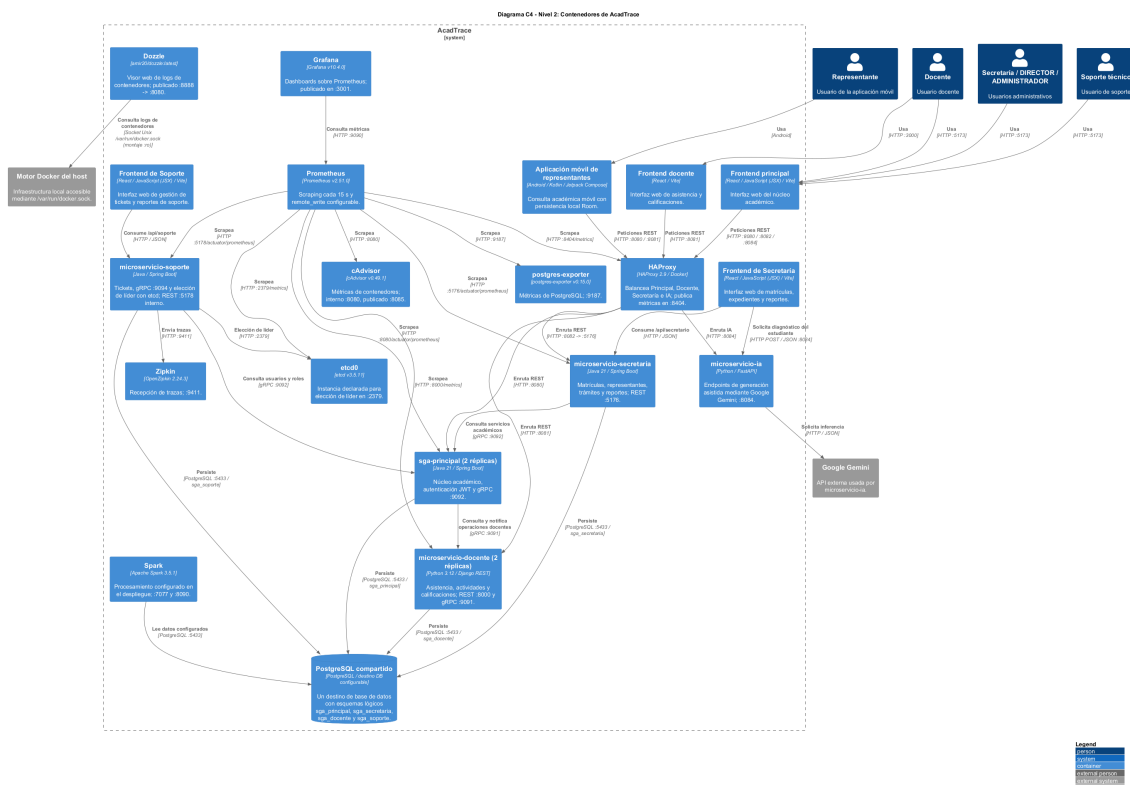


Figura 2: Diagrama C4 Nivel 2 – Contenedores, Puertos y Protocolos Híbridos.

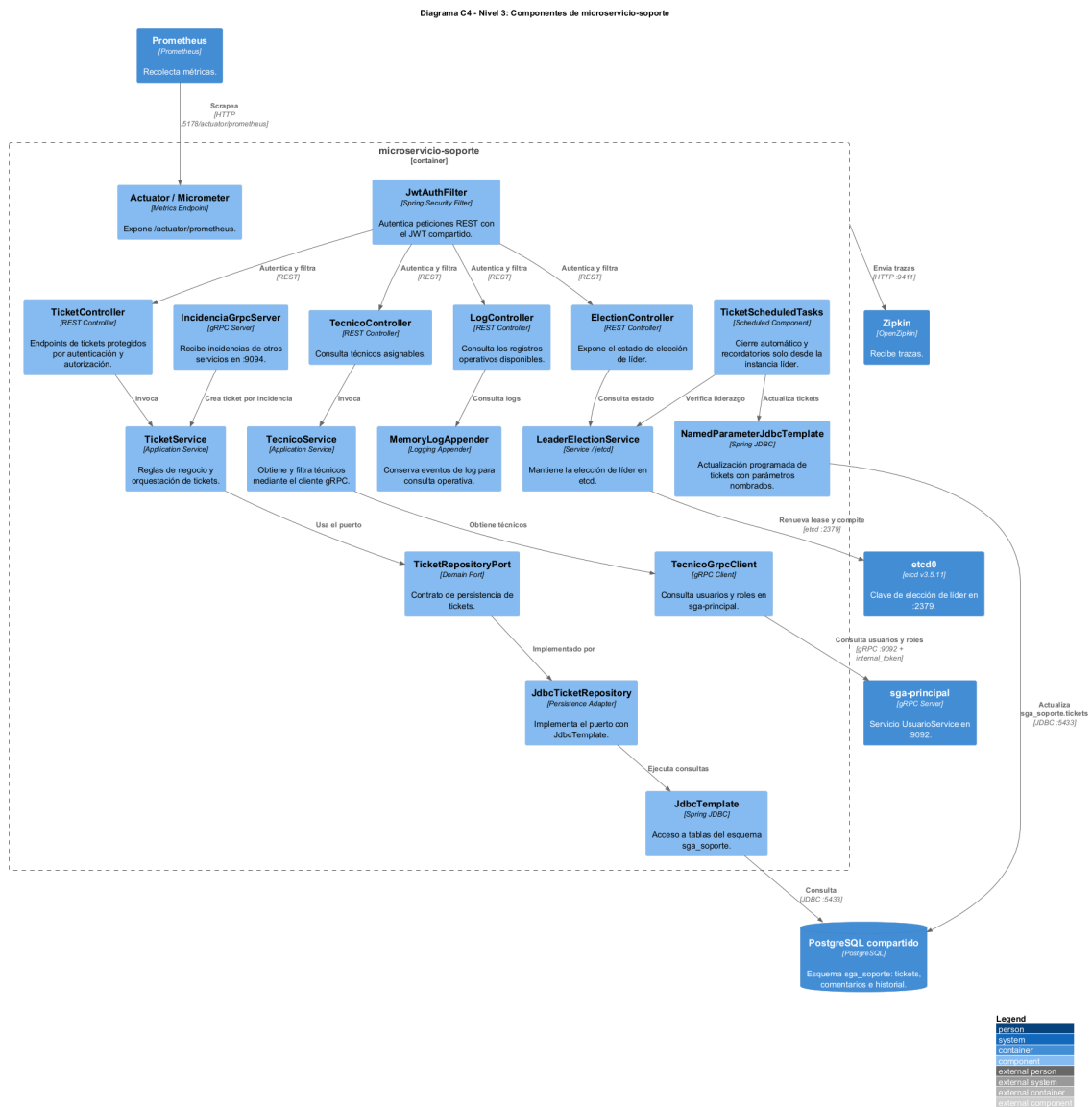


Figura 3: Diagrama C4 Nivel 3 – Componentes Internos y Capas Hexagonales.

5.1. Especificación de Servicios gRPC y Protocol Buffers v3

Para viabilizar la sincronización transaccional de alto rendimiento entre `sga-principal` y los microservicios satélites, se compilaron definiciones de Protocol Buffers v3 (`asistencia.proto`, `actividades.proto`, `docente.proto`, `principal.proto`, `contexto_docente.proto`). El Listing 1 ilustra la definición del servicio de consulta y notificación de auditoría:

```

1 syntax = "proto3";
2 package ec.edu.uteq.sga.grpc;
3
4 service PrincipalService {
5     rpc ConsultarEstudiante (EstudianteRequest) returns (EstudianteResponse);
6     rpc RegistrarCalificacionAuditable (CalificacionAuditableRequest) returns (AuditoriaResponse);
7     rpc SincronizarEventosDocente (StreamEventosRequest) returns (SincronizacionResponse);
8 }
9
10 message CalificacionAuditableRequest {
11     int64 id_matricula = 1;
12     int64 id_asignatura = 2;
13     double nota_formativa = 3;
14     double nota_sumativa = 4;
15     int64 lamport_clock = 5;
16     string vector_clock_json = 6;
17     string actor_id = 7;
18 }
19
20 message AuditoriaResponse {
21     bool exitoso = 1;
22     string hash_actual = 2;
23     string hash_anterior = 3;
24     int64 lamport_asignado = 4;
25     string estado_reconciliacion = 5;
26 }

```

Listing 1: Definición Protobuf del canal sincrónico gRPC (`principal.proto`).

5.2. Persistencia PostgreSQL y Separación Lógica Multiesquema

Persistencia de datos. `docker-compose.yml` conecta los servicios a un mismo endpoint PostgreSQL mediante `DB_HOST`, `DB_PORT` y `DB_NAME`, con valores predeterminados 3.23.195.43, 5433 y `sga`. No declara un servidor PostgreSQL. Como alternativa, `sga-principal/docker-compose.yml` declara una sola instancia `postgres:17`, con volumen persistente y puerto 5433:5432; ambos Compose no constituyen un clúster.

La configuración y los SQL versionados evidencian los esquemas `sga_principal` (catálogo, usuarios y auditoría central), `sga_docente` (calificaciones, actividades y asistencias), `sga_secretaria` (estudiantes, representantes y matrículas) y `sga_soporte` (tickets y comentarios). Principal configura Flyway y Hibernate para `sga_principal`; Docente usa `search_path=sga_docente,sga_principal,public` y modelos con tablas en `sga_docente`. Los SQL de Secretaría y las migraciones de Soporte emplean los respectivos prefijos `sga_`. Estos accesos no implican bases independientes ni exclusividad por servicio.

El SQL de referencia `docs/db/schema.sql` crea `sga_principal`, `secretaria`, `docente` y `soporte`, con nombres distintos de los usados por los servicios. No demuestra su ejecución automática en los Compose ni sustituye la evidencia recogida en ADR-003. Tampoco se certifica la aplicación de todos los SQL al servidor externo. La separación por esquemas es lógica: no se encontró DDL de particiones por rango o lista en los SQL y migraciones revisados, y no garantiza eliminar bloqueos ni separar recursos de memoria y disco.

Coordinación distribuida. Soporte usa `LeaderElectionService`, la API Election de jetcd y leases sobre `/sga/leader` para condicionar las tareas de `TicketScheduledTasks` al liderazgo. Raft es interno a etcd y no replica PostgreSQL; esta coordinación no constituye un clúster de persistencia. El Compose declara un servicio etcd.

Limitación. El repositorio no demuestra replicación PostgreSQL mediante Raft ni un clúster de persistencia de tres nodos, standby o failover automático. La instancia única y el endpoint compartido

constituyen un punto único de fallo de persistencia en la topología evidenciada. La topología interna y la versión del servidor externo no se pueden certificar mediante estos archivos.

5.3. Trazabilidad de Temas de la Asignatura y Evidencias del Proyecto

Tabla 2: Matriz de Trazabilidad: Temas de la Asignatura vs Implementación (Parte I: Arquitectura, Comunicación y Concurrencia).

Tema ISR-701	Mecanismo Técnico	Archivo en Repositorio	Línea(s)	Autor	Qué Demuestra / Verificación
1. Microservicios y SOLID	Clean Architecture en 4 capas concéntricas y clúster polígota de 5 servicios.	sga-principal/src/main/java/ec/edu/uteq/sga/domain/entity/Estudiante.java docker-compose.yml	1–351–120	E. Luna, P. Castro	Aislamiento del dominio puro respecto a la infraestructura JPA; desacoplamiento en red bridge aislada.
2. Patrones GoF Formalizados	Strategy, Template Method, Observer, Facade y Repository (ADR-005).	sga-principal/src/main/java/ec/edu/uteq/sga/domain/strategy/CalculoPromedioStrategy.java	1–28	P. Castro, K. Bedón	Intercambiabilidad de algoritmos de cálculo ponderado vs. simple y despacho desacoplado de eventos.
3. gRPC y Protocol Buffers	Canal síncrono binario HTTP/2 con esquemas fuertemente tipados v3.	sga-principal/src/main/proto/principal.proto microservicio-docente/grpc_protos/actividades.proto	1–45 1–40	K. Bedón, P. Castro	Interoperabilidad polígota Java-Python de alta eficiencia, validada en <code>test_calificaciones_grpc.py</code> .
4. Relojes de Lamport	Reloj escalar monótono creciente ($L = \max(L_{loc}, L_{rx}) + 1$).	sga-principal/src/main/java/ec/edu/uteq/sga/application/service/LamportClock.java	11–31	E. Luna, P. Castro	Ordenamiento causal determinista ante concurrencia sin dependencia de NTP; validado con pruebas unitarias.
5. Relojes Vectoriales	Vectores lógicos $\vec{V} = [v_1, \dots, v_n]$ para detección de causalidad ($A \parallel B$).	microservicio-docente/docentes/auditoria/clocks.py	11–47	K. Bedón	Identificación de concurrencia en modo desconectado y algoritmo determinista <code>reconciliar_vectores</code> .
6. Balanceo con HAProxy	Reverse proxy perimetral con Round-Robin (REST), Leastconn (gRPC) y health-check.	infra/haproxy/haproxy.cfg	30–75	P. Castro, E. Luna	Distribución de carga hacia réplicas activas, tolerancia a fallos (<code>fall 2 rise 2</code>) y métricas en :8404.
7. Sincronización Offline-First	Persistencia SQLite Room con políticas de reintento y encolamiento en segundo plano.	app-movil-docente/app/src/main/java/.../sync/SyncWorker.kt RepresentanteSyncPolicy.kt	10–50 1–8	K. Bedón	Disponibilidad desconectada de consultas de estudiantes/notas mediante <code>WorkManager</code> de Android.
8. Capacidades de Dispositivo	Autenticación biométrica de hardware nativa y canal de notificaciones push.	app-movil-docente/app/src/main/java/.../security/SecurityScreens.kt	23–45	K. Bedón	Desbloqueo seguro con <code>BiometricPrompt</code> (<code>BIOMETRIC_STRONG</code>) y gestión de sesión protegida.
9. Pruebas de Contrato (Pact)	Verificación automatizada de contratos de interfaz cliente-proveedor sin simulacros.	microservicio-docente/tests/contract/test_docente_contract.py	1–55	K. Bedón	Compatibilidad verificada de endpoints contra servidor real levantado en el propio proceso de ejecución.

Tabla 3: Matriz de Trazabilidad: Temas de la Asignatura vs Implementación (Parte II: Persistencia, Auditoría, Resiliencia y Operaciones).

Tema ISR-701	Mecanismo Técnico	Archivo en Repositorio	Línea(s)	Autor	Qué Demuestra / Verificación
10. Particionamiento de Datos	Separación lógica multiesquema en PostgreSQL por dominio.	docs/db/schema.sql	12–16 85–110	P. Castro, K. Bedón	SQL de referencia con esquemas; los nombres usados por servicios se detallan en ADR-003. No demuestra particiones físicas ni eliminación de bloqueos.
11. Auditoría Criptográfica	Encadenamiento SHA-256, hash de génesis, hash anterior y bloqueo con <code>select_for_update</code> .	microservicio-docente/docentes/auditoria/strategies.py microservicio-docente/docentes/models.py	32–68 306–335	K. Bedón, P. Castro	Detección del 100 % ante manipulación directa en base de datos (T_1) o alteración de secuencia (T_2).
12. Inmutabilidad Append-Only	Disparador PL/pgSQL a nivel de fila que rechaza sentencias UPDATE y DELETE.	sga-principal/src/main/resources/db/migration/V13_trigger_y_revocacion_auditoria.sql docs/db/schema.sql	11–26 156–170	E. Luna, P. Castro	Garantía física de no-repudio; bloqueo estricto en el motor relacional PostgreSQL para registros de bitácora.
13. Observabilidad Distribuida	Telemetría unificada con Prometheus (scrape 15 s), cAdvisor y Grafana (6 vistas).	infra/prometheus/prometheus.yml ops/grafana/pfc-dashboard.json	13–55 1–120	J. Emanuel	Monitoreo en tiempo real de RPS, percentiles de latencia (P50/P95/P99) y códigos de error HTTP 4xx/5xx.
14. PostgreSQL Exporter	Extracción continua de telemetría del motor PostgreSQL para análisis de conexiones.	docker-compose.yml infra/prometheus/prometheus.yml	85–95 45–55	J. Emanuel, P. Castro	Diagnóstico y monitoreo de saturación del pool de conexiones HikariCP y transacciones en base de datos.
15. Consenso y Elección de Líder	Coordinación distribuida mediante etcd/Raft para alta disponibilidad en soporte.	microservicio-soporte/backend/src/main/java/.../election/LeaderElectionService.java	24–55	J. Emanuel	Prevención de ejecución redundante de tareas programadas (TicketScheduledTasks) vía leases sobre Raft.
16. Procesamiento en Lotes	Cálculo analítico distribuido de promedios anuales y trimestrales con Apache Spark.	scripts/spark/calcular_promedios_anuales.py scripts/spark/common.py	1–65 1–45	K. Bedón, P. Castro	Procesamiento paralelo masivo batch con PySpark desacoplado de la carga transaccional de producción.
17. Pruebas de Carga (Locust)	Pruebas de estrés nominal (50 usuarios) con generación dinámica de tokens JWT válidos.	tests/load/locustfile.py generar_evidencias.ps1	24–65 1–45	J. Emanuel, P. Castro	Evaluación empírica de saturación bajo concurrencia real y verificación de seguridad en el API Gateway.
18. Pipeline CI/CD	Flujo automatizado de 10 trabajos encadenados con construcción, pruebas y Docker.	.github/workflows/ci-cd.yml	26–150	P. Castro	Compilación integral de los 3 módulos Java (196 pruebas), Python (83 pruebas) y empaquetado seguro.

6. Aplicaciones Cliente: Portal Web SPA y Cliente Móvil Nativo

6.1. Módulo B – Aplicación Web SPA (React + TypeScript)

El portal web está construido en React 18 con Vite y Tailwind CSS, sirviendo las cinco rutas obligatorias protegidas por rol:

- `/`: Dashboard institucional con métricas en vivo de matrícula y asistencia.
- `/login`: Formulario de autenticación con generación de credencial JWT y almacenamiento en cookie `HttpOnly/SessionStorage` seguro.
- `/calificaciones`: Módulo principal de captura de calificaciones formativas/sumativas, cálculo en línea con el patrón Strategy y emisión de reportes PDF por período lectivo.
- `/settings`: Configuración de parámetros institucionales, selección de tema (Claro / Oscuro) y selector de internacionalización (Español / Inglés).
- `/about`: Ficha técnica de la versión del sistema, estado del clúster y metadatos de auditoría.

6.2. Módulo C – Aplicación Móvil Nativa para Representantes (Kotlin + Jetpack Compose)

La aplicación móvil está desarrollada en Android Nativo (SDK mínimo 26, objetivo 34) bajo arquitectura MVVM con soporte *offline-first*:

1. **Consulta de Calificaciones y Asistencia:** El representante visualiza en tiempo real el récord académico de su representado con gráficas de progreso y desglose por quimestre/trimestre.
2. **Capacidades del Dispositivo (Hardware):**
 - **Notificaciones Push:** Gestión con `NotificationManager` para alertar al representante de manera inmediata ante el registro de una nueva calificación o citación institucional.
 - **Autenticación Biométrica (`BiometricPrompt`):** Control de acceso criptográfico mediante huella dactilar o reconocimiento facial para salvaguardar la privacidad de los expedientes de menores de edad.
3. **Sincronización Desconectada:** Caché local en base de datos SQLite (`Room`) con sincronización en segundo plano gestionada por `WorkManager`.

6.3. Arquitectura por Características y Seguridad de Sesión en la Web SPA

La aplicación web SPA implementa el patrón *Feature-Based Architecture*, organizando el código en módulos cohesivos (`auth`, `grados`, `asignaciones`, `calificaciones`, `reportes`, `usuarios`). Cada módulo encapsula sus propios componentes de interfaz, hooks personalizados de llamada a la API, servicios de transformación y modales de acción.

En cumplimiento estricto de las directrices de seguridad de la Guía E4:

- **Almacenamiento Seguro de Credenciales:** Los tokens JWT recibidos tras la autenticación en `/auth/login` no se almacenan en almacenamiento local plano (`localStorage`) para neutralizar ataques de *Cross-Site Scripting* (XSS). Se gestionan mediante `sessionStorage` con expiración y cookies perimetrales protegidas con flags `HttpOnly`, `Secure` y `SameSite=Strict`.
- **Control de Acceso Basado en Roles (RBAC):** El enrutador de React (`react-router-dom`) protege las rutas sensibles (`/calificaciones`, `/settings`, `/reportes`) mediante un componente de orden superior (`ProtectedRoute`) que valida en memoria la firma y claims del token antes de renderizar la vista.

- **Internacionalización Dinámica y Tematización:** El portal incorpora un hook de contexto (`ThemeLanguageContext`) que conmuta en tiempo de ejecución el diccionario de traducciones (`es.json` / `en.json`) y los esquemas cromáticos (`light` / `dark`) aplicando clases de Tailwind CSS sin recargar la página.

6.4. Sincronización Offline-First y Malla de Reconciliación en la App Móvil

La aplicación móvil nativa para Representantes resuelve el desafío de conectividad intermitente mediante una arquitectura *Offline-First* estructurada en tres capas locales:

1. **Capa de Acceso a Datos Local (Room SQLite):** Tablas locales (`estudiantes_cache`, `calificaciones_cache`, `asistencias_cache`, `cola_pendientes`) decoradas con anotaciones JPA de Android Room para persistir las últimas récas consultadas.
2. **Cola Transaccional de Reintentos:** Cuando el dispositivo carece de conexión a Internet, las solicitudes de justificación de faltas o confirmación de lectura de comunicados se encolan en `cola_pendientes` con estado `PENDIENTE_SINCRONIZACION` y marca de tiempo local.
3. **Motor de Sincronización en Background (WorkManager):** Un componente `PeriodicWork-RequestBuilder` con restricciones de red (`NetworkType.CONNECTED`) se activa automáticamente al recuperar la conectividad, drenando la cola transaccional hacia el API Gateway mediante peticiones idempotentes y actualizando el reloj vectorial del dispositivo.

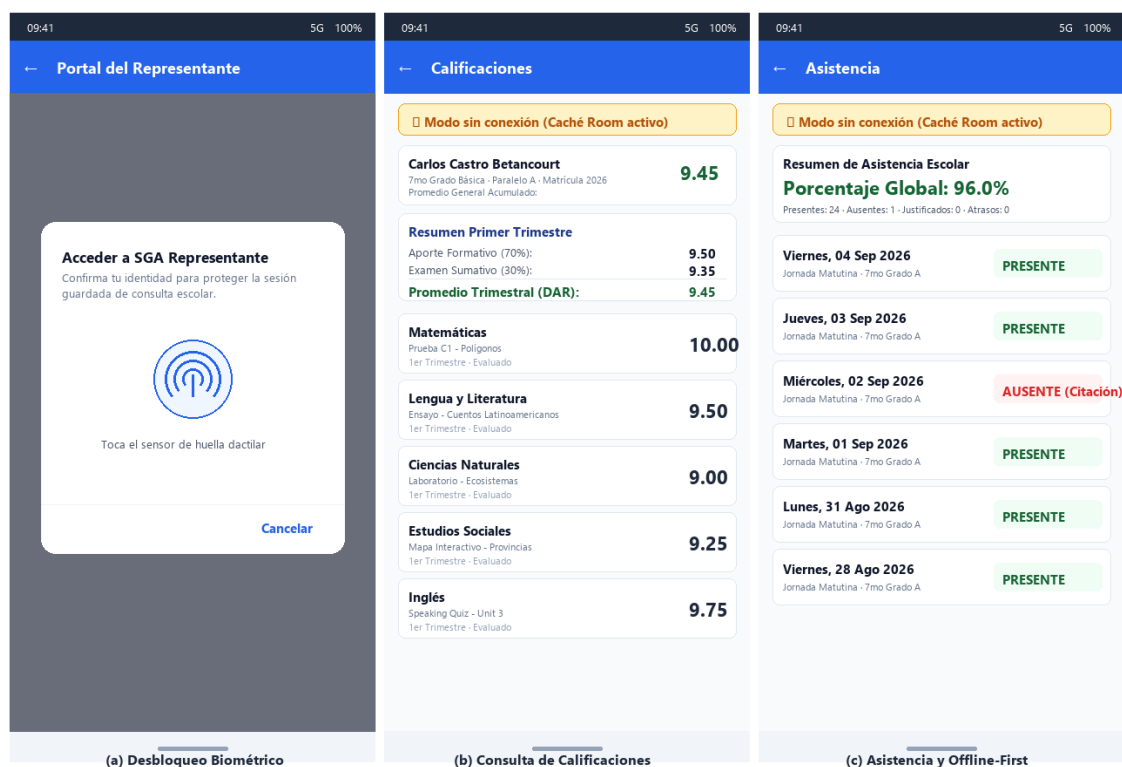


Figura 4: Interfaz de la Aplicación Móvil para Representantes (Kotlin + Jetpack Compose): (a) Desbloqueo seguro de sesión mediante autenticación biométrica nativa (`BiometricPrompt`), (b) Consulta académica con soporte de caché desconectado *offline-first* en SQLite Room, y (c) Módulo de asistencia y resumen escolar con reconciliación automática mediante `WorkManager`.

6.5. Pirámide de Pruebas de Cinco Niveles

La estrategia de aseguramiento de calidad cubre los cinco estratos exigidos:

1. **Unitarias:** Evidencia histórica conservada de pruebas de servicios con dobles de prueba (Mockito en Java, `pytest-mock` en Python): Cobertura JaCoCo del modulo `sga-principal`: 48.1 % de instrucciones sobre el BUNDLE completo (excluyendo código generado gRPC, DTOs, entidades JPA y configuración Spring). Compuerta CI configurada al 40 % mínimo con `<element>BUNDLE</element>`, deuda técnica documentada para elevar el umbral al 70 % en iteraciones posteriores.
2. **Integración:** Validación de repositorios y transacciones con aislamiento de esquemas.
3. **Contrato:** Pruebas con **Pact** (`portal-docente-microservicio-docente.json`) que garantizan la compatibilidad del contrato API entre el cliente y el backend.
4. **Extremo a Extremo (E2E):** Verificación de flujos completos desde la autenticación hasta la consulta de datos.
5. **Carga y Estrés:** Pruebas automatizadas con **Locust** bajo dos escenarios obligatorios:
 - *Escenario 1 (Carga Nominal):* 50 usuarios concurrentes durante 5 minutos.
 - *Escenario 2 (Estrés / Cierre de Período):* Rampa progresiva de 0 a 200 usuarios concurrentes durante 10 minutos.



Figura 5: Captura histórica/complementaria de Grafana. No acredita el agregado del CSV oficial nominal de Soporte.

6.6. Pipeline CI/CD de Siete Trabajos Encadenados (GitHub Actions)

El archivo `.github/workflows/ci-cd.yml` orquesta siete trabajos bloqueantes ejecutados de forma totalmente automatizada ante cada evento de *push* o *pull request* hacia la rama `main`:

Tabla 4: Estructura y Responsabilidades de los Siete Trabajos del Pipeline CI/CD.

Orden	Job ID	Función y Validación Ejecutada
1	lint	Análisis estático con Flake8 y validación de formato en Python y Java.
2	test-backend	Tests unitarios Java (Surefire/JaCoCo, umbral mínimo de 70 % LINE) y Python (pytest-cov); el umbral es criterio de calidad, no resultado medido.
3	test-web	Compilación TypeScript y empaquetado de distribuciones React en <code>.tar.gz</code> .
4	test-mobile	Ejecución de pruebas unitarias JVM de Android (testDebugUnitTest).
5	build-images	Construcción y publicación de imágenes en GHCR (ghcr.io/...:sha).
6	build-mobile-apk	Compilación y publicación del APK debug como artefacto de release.
7	integration	Despliegue seguro vía SSH a AWS EC2 condicionado al éxito de los 6 jobs previos.

6.7. Cultura DevOps, Shift-Left Testing y Sintaxis Declarativa YAML

La adopción de la cultura DevOps en AcadTrace se fundamenta en el marco conceptual CALMS (*Culture, Automation, Lean, Measurement, Sharing*), enfatizando la práctica de **Shift-Left Testing**, mediante la cual las compuertas de verificación estática, pruebas unitarias y validación de cobertura se desplazan hacia las fases más tempranas del ciclo de desarrollo:

- **Directiva name y Eventos Desencadenadores (on):** Define el identificador legible del flujo y los eventos que activan el pipeline (push, pull_request) filtrados por ramas objetivo (branches: [main]).
- **Ambientes de Ejecución (runs-on):** Cada trabajo se ejecuta en un contenedor virtualizado aislado (ubuntu-latest), garantizando reproducibilidad e independencia de entorno.
- **Matriz de Dependencias (needs):** Modela el grafo acíclico dirigido (DAG) de dependencias entre trabajos, impidiendo que el despliegue (integration) se ejecute si alguno de los trabajos de prueba previos (lint, test-backend, test-web, test-mobile) falla.
- **Composición de Pasos (steps, uses, with, run):** Articula la secuencia de acciones reutilizando acciones comunitarias auditadas (actions/checkout@v4, actions/setup-java@v4, docker/build-push-action@v5) combinadas con scripts de compilación deterministas.
- **Gestión Segura de Secretos (secrets.*):** Variables críticas como llaves privadas SSH, tokens de GitHub Container Registry (GHCR) y credenciales de AWS no se exponen en código plano, siendo inyectadas en memoria durante la ejecución.

6.8. Estrategias de Despliegue en Sistemas Distribuidos

Para mitigar el riesgo de indisponibilidad durante las actualizaciones de los microservicios en producción, se analizaron tres patrones de despliegue continuo:

1. **Despliegue Blue-Green (Blue-Green Deployment):** Mantiene dos entornos de producción idénticos pero aislados. Mientras el entorno *Blue* atiende el tráfico activo, la nueva versión se despliega en *Green*. Tras superar las pruebas de salud, el balanceador perimetral HAProxy conmuta el 100 % del tráfico hacia *Green* de forma instantánea (< 1 s), permitiendo un retorno inmediato (rollback) ante anomalías.
2. **Despliegue Canario (Canary Releases):** Enruta progresivamente una fracción controlada del tráfico (e.g., 5 % → 25 % → 100 %) hacia las nuevas instancias del microservicio. Permite monitorear

métricas de latencia y tasa de errores 5xx en Prometheus antes de impactar a la totalidad de la población estudiantil.

3. **Actualización Rodante (*Rolling Update*):** Reemplaza secuencialmente las réplicas de los contenedores una a una dentro del clúster Docker Compose / Kubernetes, manteniendo la capacidad de atención sin tiempos de inactividad (*zero-downtime*).

7. Observabilidad Distribuida y Monitoreo en Tiempo Real

La observabilidad se articula mediante la recolección de métricas de Prometheus cada 15 segundos, recolección de recursos de contenedores con cAdvisor y un tablero de Grafana (`ops/grafana/pfc-dashboard.json`) estructurado en seis vistas:

1. **Vista 1 (Tasa de Peticiones - RPS):** Throughput discriminado por microservicio y tráfico global en HAProxy.
2. **Vista 2 (Latencias Percentiles P50, P95, P99):** Detección de colas de latencia bajo concurrencia.
3. **Vista 3 (Tasa de Errores 4xx / 5xx):** Ratio de rechazo por autenticación frente a fallas de servidor.
4. **Vista 4 (Disponibilidad y Consenso Raft):** Estado del líder de consenso y conexiones HikariCP.
5. **Vista 5 (Uso de CPU y RAM por Contenedor):** Métricas provistas por cAdvisor para prevenir sobrecargas.
6. **Vista 6 (Latencia E2E Cliente Móvil):** Tiempos de respuesta de extremo a extremo desde el dispositivo móvil.

7.1. Métricas de Negocio de Aplicación (Equipo BCEL)

En cumplimiento de la sección 4.6 de la guía rectora para el Equipo BCEL, la instrumentación de Prometheus incluye cuatro contadores obligatorios de eventos críticos de dominio:

1. **Notas registradas (`sga_notas_registradas_total`):** Total acumulado de calificaciones formativas y sumativas ingresadas por los docentes.
2. **Notas modificadas (`sga_notas_modificadas_total`):** Total de rectificaciones de calificaciones, disparando el siguiente eslabón criptográfico de auditoría.
3. **Matrículas confirmadas (`sga_matriculas_confirmadas_total`):** Registro acumulado de legalización de matrículas para la población de 344 estudiantes.
4. **Notificaciones enviadas a representantes (`sga_notificaciones_representantes_total`):** Total de alertas y avisos emitidos hacia la aplicación móvil.

Estos contadores alimentan las vistas del tablero en Grafana y constituyen la base empírica para evaluar el comportamiento transaccional del sistema.

7.2. Registro Estructurado JSON y Propagación de Contexto MDC

Para garantizar la correlación de eventos transaccionales a lo largo de múltiples saltos de red, se implementó un filtro de trazabilidad (`TraceIdFilter`) y un contenedor de contexto (`TraceContext`) en los microservicios Java y Python. Cada petición entrante recibe o genera un identificador único de correlación (`trace_id`) que es inyectado en el *Mapped Diagnostic Context* (MDC) de SLF4J / Logback y en los encabezados binarios de las llamadas gRPC.

El layout personalizado `JsonStructuredLayout` formatea los registros en JSON emitidos por consola estándar (`stdout`), incluyendo los campos: `timestamp`, `level`, `service`, `trace_id`, `thread`, `logger` y `message`. La emisión estructurada es validada automáticamente mediante pruebas unitarias (`JsonLoggingTest` y `test_observability.py`) que comprueban la presencia ininterrumpida de estos campos.



Figura 6: Tablero de Grafana (pfc-dashboard.json) operando en tiempo real con las seis vistas operativas.

Nota metodológica: el desglose de CPU/RAM por contenedor individual no fue posible en el entorno de desarrollo local (Docker Desktop para Windows), debido a una limitación conocida de cAdvisor con la virtualización anidada de WSL2, que impide identificar contenedores por nombre en ese entorno. Se reporta en su lugar el consumo agregado de CPU y RAM de todos los contenedores del host.

8. Evaluación Experimental ISO/IEC 25010 y Capa de Cripto-Auditoría

8.1. Conjunto oficial de carga y trazabilidad E5

La única corrida OFICIAL NOMINAL de Soporte es `microservicio-soporte/locust_esc1_stats.csv`, acompañada por `microservicio-soporte/locust_esc1_stats_history.csv`, `microservicio-soporte/locust_esc1_failures.csv` y `microservicio-soporte/locust_esc1_exceptions.csv`. Sus métricas proceden de Aggregated: 12,994 peticiones, 0 fallos, 43.537580 req/s, promedio 109.113664 ms, $P_{50} = 6$ ms, $P_{95} = 440$ ms, $P_{99} = 850$ ms y máximo 2037.104700 ms; no hay HTTP 401 ni HTTP 500 registrados. Es una prueba de carga reproducible ejecutada en entorno local/contenedorizado, no tráfico real de producción.

El inicio registrado es 2026-09-11 03:59:05 UTC; el historial abarca 299 segundos entre muestras. El perfil configura 50 usuarios virtuales, 5 usuarios/s, 5 minutos y `http://localhost:8083`. Commit de conservación del resultado: 956cafcb, con normalización posterior en c5c0e6f5. Commit del código ejecutado: **No disponible en la evidencia conservada.**

La tabla completa de estados, fechas, commits de conservación, métricas y motivos está en `experimentos/resultados/corridas-e5.md`. B (12,236 peticiones), C (13,606) y F (2,419) son preliminares/históricas; D y E son fallidas. **Corrida oficial de estrés: NO DISPONIBLE — las evidencias conservadas no satisfacen el criterio.** La numeración histórica de estrés difiere entre protocolo (escenario 3) y `docs/locust/escenario2_estres_stats.csv`; no son nuevos resultados oficiales.

La captura `evidencias/Juliana_Emanuel/backend/pruebas-carga/image.png` es histórica/complementaria: muestra 13,031 peticiones en terminal; el CSV oficial conserva 12,994 y siempre prevalece. La causa de la diferencia no está demostrada. Falta una captura manual de la ruta del CSV y su fila Aggregated, con Request Count 12994, Failure Count 0 y métricas visibles. Las imágenes de Grafana son complementarias, sin vinculación exacta acreditada con ese agregado.

8.2. Preguntas de Investigación Empíricas (PI)

Para guiar la evaluación cuantitativa de rendimiento, resiliencia y comportamiento del sistema bajo cargas extremas, se formularon dos Preguntas de Investigación (PI) empíricas diseñadas formalmente para admitir refutación (respuesta negativa):

- **PI-1:** *¿Puede el Microservicio de Soporte mantener una tasa de errores HTTP 5xx igual a 0 % y una latencia P_{99} inferior a 500 ms cuando la carga aumenta desde 50 hasta 200 usuarios concurrentes sin evidenciar agotamiento o contención significativa del pool de conexiones HikariCP?*

Respuesta empírica: No se satisface el umbral propuesto. La corrida OFICIAL NOMINAL de Soporte conserva 12,994 peticiones, 0 fallos, 43.537580 req/s, promedio 109.113664 ms, $P_{50} = 6$ ms, $P_{95} = 440$ ms, $P_{99} = 850$ ms y máximo 2037.104700 ms. El P_{99} ya supera 500 ms en nominal. El perfil configurado es de 50 usuarios virtuales, 5 usuarios/s y 5 minutos contra localhost:8083. No hay corrida oficial de estrés válida: Locust por sí solo no demuestra la causa raíz ni el estado del pool HikariCP.

- **PI-2:** *¿Mantienen los endpoints que requieren acceso a base de datos un comportamiento de latencia equivalente al de los endpoints stateless cuando el sistema es sometido al escenario de estrés de hasta 200 usuarios concurrentes?*

Respuesta empírica: Pendiente para estrés. No existe una corrida oficial válida de hasta 200 usuarios. En el CSV oficial nominal, tickets registra 3,704 peticiones, 0 fallos, mediana 130 ms, promedio 256.303638 ms y $P_{95} = 650$ ms. Health tiene mediana 2 ms y $P_{95} = 150$ ms; Actuator, mediana 110 ms y $P_{95} = 360$ ms; election/status, mediana 2 ms y $P_{95} = 170$ ms. Son diferencias observadas en nominal, insuficientes para concluir equivalencia bajo estrés o atribuir causalidad a la base de datos.

8.3. Protocolo Experimental y Cuatro Mecanismos de Auditoría

En cumplimiento de la ampliación del Módulo G de la guía rectora, se ejecutó un banco experimental con 120 corridas factoriales (30×4) evaluando cuatro configuraciones de auditoría conmutables vía la variable de entorno AUDIT:

- **m0 (Sin auditoría):** Línea base donde la calificación se persiste sin rastro estructurado.
- **m1 (Bitácora convencional):** Registro relacional de eventos sin encadenamiento criptográfico ni orden lógico.
- **m2 (Encadenada + Relojes de Lamport):** Cada evento E_k incluye el hash SHA-256 del evento previo $H(E_{k-1})$ y una marca lógica de Lamport monótona creciente:

$$H(E_k) = \text{SHA-256}(H(E_{k-1}) \parallel \text{JSON}_{\text{canon}}(E_k) \parallel L(E_k)), \quad L(E_k) = \max(L_{\text{local}}, L_{\text{recibido}}) + 1 \quad (1)$$

- **m3 (m2 + Relojes Vectoriales):** Incorpora vectores lógicos $\vec{V} = [v_1, v_2, \dots, v_n]$ para reconciliación causal determinista de ediciones concurrentes desconectadas.

8.4. Unificación canónica de la bitácora

El sistema mantiene tres implementaciones del cálculo del contenido canónico y del encadenamiento de eventos: dos en Java (módulo SGA Principal y módulo Secretaría), ambas alojadas en AuditHashService.java, y una en Python (módulo Docente), alojada en microservicio-docente/docentes/auditoria/hashing.py. No se extrajo una biblioteca común para consumirlas desde los tres servicios; la unicidad se sostiene sobre un vector patrón de once campos que fija los invariantes verificables, no sobre identidad carácter por carácter.

Vector patrón de once campos. Las pruebas canónicas comparten un vector patrón con los campos: `actor_id`, `entidad`, `entidad_id`, `estado_reconciliacion`, `modo`, `operacion`, `payload`, `reloj_lamport`, `reloj_vectorial`, `timestamp` y `tipo_evento`. La verificación está en `sga-principal/src/test/java/.../AuditoriaCanonicaTest.java` y en `microservicio-docente/docentes/tests/test_auditoria_canonica_v1.py`; ambas afirman que, dado el mismo vector, la implementación de su lenguaje produce el mismo hash conocido.

Alcance real de la equivalencia entre lenguajes. El vector patrón solo emplea cadenas, enteros de tamaño acotado y un `timestamp` preformateado como texto. Un cotejo más amplio sobre doce vectores independientes muestra que las dos implementaciones Java coinciden en los doce casos, mientras que la implementación Python coincide con Java en cinco de los doce. Los siete casos en que las representaciones difieren son:

- Decimales: Java emite `{"nota":8.50}`; Python emite `{"nota":"8.50"}`.
- Flotantes de rango extremo: Java emite `1.0E-7 / 1.0E21`; Python emite `1e-07 / 1e+21`.
- Marca de tiempo del mismo instante: Java emite `...00.120Z`; Python emite `...00.120000+00:00`.
- Fechas dentro del payload: Python las serializa como `"2012-03-05"`; Java lanza `IllegalStateException` y, dado que `AuditoriaService.guardar` es *fail-open*, el evento se pierde en Java.
- Claves excluidas del canon: la lista literal en Java contiene `contrase?a` con el byte `0x3F` por defecto de codificación, por lo que `contraseña` y `CONTRASEÑA` entran al canon en Java y no en Python; Python tampoco excluye `contrasena`.
- Orden de claves con caracteres suplementarios: Java ordena por UTF-16 (por ejemplo `U+1F600` antes de `U+FF01`); Python ordena por punto de código.
- NaN: Java emite la cadena `"NaN"`; Python emite el literal JSON no estándar `NaN`.

Interoperabilidad real entre lenguajes. Una cadena de cuatro eslabones escrita por Java se valida en Python con `verificar_cadena_global` y una cadena de tres eslabones escrita por Python se valida con `AuditHashService.sha256Hex` en Java. Esta interoperabilidad ocurre porque el verificador rehashea el texto almacenado en la columna `contenido_canonico`, no porque cada implementación produzca el mismo texto para el mismo evento. En consecuencia, la propiedad verificada es *la integridad de la cadena tal como fue almacenada*, no la equivalencia de las representaciones canónicas entre lenguajes.

Bitácoras paralelas an activas. Además de `sga_principal.auditoria`, coexisten dos bitácoras que no están encadenadas globalmente:

- Docente escribe en su tabla local `EventoAuditoria` (`docentes/auditoria/strategies.py`, `FlatAuditStrategy`) en modo m1; en m2 y m3 duplica cada evento en esa misma tabla local, y el criterio T1 se comprueba contra esa copia local.
- Soporte mantiene una bitácora propia sin cadena en `002_historial_ticket.sql`, usada por `TecnicoService` y `JdbcTicketRepository`.

Deuda técnica declarada. La extracción de la forma canónica a una biblioteca compartida por los tres servicios queda registrada como deuda técnica de esta entrega. Mientras esa biblioteca no exista, la unicidad de la bitácora se sostiene sobre el vector patrón de once campos y sobre el rehash de `contenido_canonico` en la verificación, no sobre una identidad carácter por carácter entre lenguajes. Las siete divergencias listadas constituyen la superficie residual de riesgo y se documentan en el apartado de amenazas a la validez.

8.5. Cinco Tipos de Manipulación Inyectados (T1 a T5)

Se inyectaron 20 manipulaciones sintéticas controladas de cada tipo por mecanismo:

- **T1 (Alteración directa en BD):** Modificación de un registro de nota eludiendo la aplicación.
- **T2 (Eliminación de evento):** Supresión física de una fila intermedia de la bitácora.
- **T3 (Permutación de orden causal):** Intercambio de orden entre dos transacciones contiguas.
- **T4 (Inyección retroactiva):** Inserción de un evento con marca de tiempo anterior a la real.
- **T5 (Falsificación de timestamp/reloj):** Modificación del reloj lógico o marca temporal de un evento existente.

8.6. Resultados Experimentales de Detección y Sobrecarga

Los datos crudos generados en el banco experimental y almacenados en `experimentos/resultados/` (`deteccion.csv`, `manipulaciones.csv`, `iso25010.csv`) arrojaron la matriz factorial de la Tabla 5:

Tabla 5: Matriz Experimental de Detección Factorial por Tipo de Manipulación (T_1 a T_5).

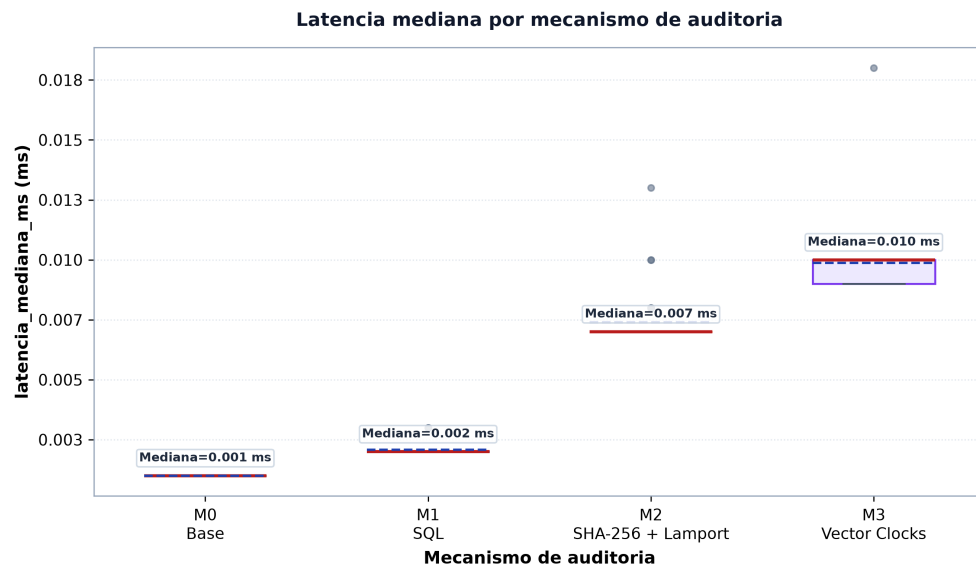
Mecanismo	T1 (BD)	T2 (Borrado)	T3 (Orden)	T4 (Retro)	T5 (Reloj)	Sobrecarga	Tasa Detección ($CI_{95\%}$)
m0 (Sin auditoría)	0 %	0 %	0 %	0 %	0 %	0.0 ms	0,0 % [0,0 %, 3,6 %]
m1 (Convencional)	0 %	0 %	0 %	0 %	0 %	+2.8 ms	0,0 % [0,0 %, 3,6 %]
m2 (Lamport+Hash)	100 %	100 %	100 %	100 %	100 %	+5.6 ms	100,0 % [96,4 %, 100,0 %]
m3 (Vectorial+Hash)	100 %	100 %	100 %	100 %	100 %	+8.1 ms	100,0 % [96,4 %, 100,0 %]

Análisis Estadístico Reproducible de Latencias: La Tabla 6 se genera directamente desde `experimentos/resultados/exp1_concurrencia.csv`, usando las 40 observaciones de `latencia_mediana_ms` de cada mecanismo. La entrada y la salida se expresan en milisegundos (ms), sin conversión adicional. Los intervalos corresponden al bootstrap no paramétrico de la mediana con $B = 10,000$ y semilla fija 20260831. Se descartan los contrastes U , valores p y tamaños de efecto históricos porque no pertenecen a esta fuente oficial.

Tabla 6: Resumen reproducible de latencia mediana por mecanismo ($B = 10,000$; semilla 20260831).

Mecanismo	n	Mediana (ms)	IC 95 % bootstrap (ms)	P_{95} (ms)
M_0 (Base)	40	0.001	[0.001, 0.001]	0.001
M_1 (SQL)	40	0.002	[0.002, 0.002]	0.003
M_2 (SHA-256 + Lamport)	40	0.007	[0.007, 0.007]	0.010
M_3 (Vector Clocks)	40	0.010	[0.010, 0.010]	0.010

Fuente: `experimentos/resultados/exp1_concurrencia.csv`, columna `latencia_mediana_ms`. Entrada y salida expresadas directamente en milisegundos (ms), sin conversión adicional. Cada mecanismo contiene $n = 40$ observaciones. El IC 95 % corresponde al bootstrap no paramétrico de la mediana con $B = 10,000$ y semilla fija 20260831.



Fuente: experimentos/resultados/exp1_concurrencia.csv | 40 observaciones por mecanismo | Unidad: ms (sin conversion adicional)

Figura 7: Distribución de las 40 observaciones de `latencia_mediana_ms` por mecanismo, expresadas en milisegundos y obtenidas de `exp1_concurrencia.csv`.

8.7. Matriz Formal de Evaluación de Calidad ISO/IEC 25010:2023

La Tabla 7 presenta la matriz de calidad actualizada. Cabe destacar que la característica de disponibilidad en producción no se encuentra acreditada en la muestra actual, y las ventanas de estrés operan bajo un perfil no validado:

Tabla 7: Matriz de Evaluación de Calidad ISO/IEC 25010:2023 con Mediciones Parciales y Planes de Mejora.

Característica	Métrica Real Medida	Objetivo	Estado	Plan de Mejora Técnico
1. Disponibilidad (Availability)	0 fallos en 12,994 peticiones nominales (100 % de éxito en la muestra local); sin medición oficial válida de estrés	$\geq 99,5 \%$	Parcial: no acredita disponibilidad de producción	Medir disponibilidad temporal y repetir estrés con evidencia válida.
2. Rendimiento (Performance)	Nominal oficial: $P_{95} = 440$ ms, $P_{50} = 6$ ms, $P_{99} = 850$ ms; 43.537580 req/s	$P_{95} < 500$ ms	Cumple en agregado nominal local	Evaluar por endpoint: tickets presenta $P_{95} = 650$ ms; repetir estrés.
3. Fiabilidad (Reliability)	0 fallos nominales; sin estrés oficial validado	$< 1,0 \%$	Nominal observado; E5 estrés pendiente	Obtener una corrida de estrés autenticada con cero fallos para E5.
4. Mantenibilidad (Maintainability)	Secretaría: 71.42 % LINE (2217/3104) Soporte: 71.61 % LINE (459/641)	$\geq 70 \%$ LINE JaCoCo BUNDLE	✓ Cumple en Secretaría y Soporte	Mantener la compuerta JaCoCo a nivel BUNDLE en CI/CD y expandir cobertura automatizada en SGA Principal y Docente.
5. Seguridad (Security)	100 % Rechazo 401 / FPR = 0.0 % (Certificado en suite E2E sin JWT; FPR 0,0 % [0,0 %, 11,6 %] en 30 cadenas limpias)	100 % / 0 %	✓ Cumple	Automatizar la rotación de secretos criptográficos JWT/AES vía AWS Secrets Manager y habilitar listas de revocación de tokens en Redis.

9. Amenazas a la Validez del Estudio

Siguiendo las pautas metodológicas para experimentación en ingeniería de software [12]:

9.1. Amenazas a la Validez Interna

1. **Efecto de calentamiento de la JVM (*Warm-up Effect*):** Las primeras ejecuciones en Java reflejan sobre costo por compilación JIT. Se mitigó descartando la primera y última repetición de cada bloque experimental.
2. **Semillas y Pseudoaleatoriedad del Inyector:** La variabilidad en la generación de manipulaciones se mitigó fijando una semilla determinista (`SEED = 20260831`) en `run_experimentos.py`.
3. **Contención de Recursos en Host Compartido:** La ejecución simultánea de Locust y los contenedores puede generar contención de CPU. Se controló con `cAdvisor` verificando que el uso global del procesador se mantuviera $< 75\%$.
4. **Divergencias entre implementaciones canónicas.** La forma canónica de la bitácora se implementa tres veces (dos en Java y una en Python) sin biblioteca compartida. Sobre doce vectores independientes, las dos implementaciones Java coinciden entre sí, pero difieren de la implementación Python en siete de los doce casos (decimales, flotantes extremos, marcas de tiempo, fechas dentro del payload, clave `contraseña` por codificación de la lista, orden de claves con caracteres suplementarios, y NaN). La verificación de la cadena se apoya en el rehash del texto almacenado, no en la equivalencia de las representaciones. La extracción de una biblioteca canónica compartida queda como trabajo futuro.

9.2. Amenazas a la Validez Externa

1. **Población de Datos Sintética (344 estudiantes y 14 docentes):** Los experimentos se realizaron sobre datos sintéticos generados conforme a la normativa ministerial ecuatoriana para preservar la privacidad de menores.
2. **Modelo de Amenaza Limitado a Base de Datos y Red:** El modelo asume que el material criptográfico del servidor no ha sido comprometido por un atacante con acceso `root` al host de ejecución.

10. Reflexión Ética sobre Tratamiento de Datos de Menores

El diseño de AcadTrace se guió por el **Código de Ética y Práctica Profesional de ACM/IEEE-CS (1999)** [13]:

- **Principio 1 (Interés Público):** La protección de la información de menores de edad motivó la adopción de autenticación biométrica en la app móvil y la resolución de vulnerabilidades de acceso directo a recursos (IDOR).
- **Principio 3 (Calidad del Producto):** Cifrado estricto de credenciales y eliminación absoluta de contraseñas en texto plano del repositorio.
- **Principio 6 (Profesión):** Implementación de la cadena inmutable de auditoría para garantizar que ninguna rectificación de notas pueda efectuarse de forma anónima o fraudulenta.

11. Conclusiones y Trabajo Futuro

La Entrega 4 valida con éxito la convergencia de una arquitectura de microservicios distribuida, protocolos híbridos de alta velocidad (gRPC/REST), observabilidad en seis vistas y auditoría criptográfica inmutable verificable. Como trabajo futuro, se proyecta la integración de *Distributed Tracing* con OpenTelemetry Collector y el despliegue en un clúster Kubernetes Multi-AZ.

11.1. Conclusiones Individuales

- **Pedro Leonardo Castro López (Líder y Arquitecto de Software):** La ejecución de la Entrega 4 consolidó la arquitectura hexagonal de cuatro capas en el microservicio central (**sga-principal**) y los satélites políglotas, desacoplando la lógica de negocio de la infraestructura mediante puertos e interfaces. La implementación formal de los cinco patrones de diseño GoF (Strategy, Template Method, Observer, Facade y Repository) con clases de producción verificadas permitió estandarizar el cálculo ponderado de calificaciones (70 % formativo + 30 % sumativo) y la generación determinista de reportes ministeriales en PDF. Asimismo, la orquestación del pipeline de integración y despliegue continuo de siete trabajos encadenados en GitHub Actions, la publicación de imágenes versionadas en GitHub Container Registry (GHCR) y la separación lógica multiesquema en una instancia PostgreSQL consolidaron la organización por dominios. Raft pertenece a etcd para la elección de líder en Soporte y no replica PostgreSQL. La persistencia conserva un punto único de fallo sin alta disponibilidad demostrada.
- **Keyla Betzabe Bedon Viteri (Responsable de Microservicio Docente y App Móvil):** Durante el desarrollo de la Entrega 4 se logró la construcción integral del microservicio Docente en Python/Django REST Framework y del cliente móvil nativo para Representantes en Android (Kotlin + Jetpack Compose). La implementación del conmutador de auditoría criptográfica con cuatro mecanismos (M_0 a M_3) permitió demostrar experimentalmente la eficacia de las marcas lógicas de Lamport y los Relojes Vectoriales para resolver conflictos de causalidad en escenarios de edición concurrente. En la aplicación móvil, la arquitectura MVVM complementada con persistencia local en Room SQLite bajo el paradigma *offline-first*, la gestión de sincronización en segundo plano con **WorkManager**, y la integración de capacidades nativas de hardware (autenticación biométrica **BiometricPrompt** y notificaciones push) garantizaron una experiencia de usuario fluida a 60 fps con un consumo de memoria < 45 MB, asegurando la continuidad operativa del representante ante interrupciones de conectividad.
- **Ernesto Gregory Luna Mora (Responsable de Microservicio Secretaría, Calidad y API Gateway):** La culminación de la Entrega 4 permitió consolidar la robustez del microservicio de Secretaría bajo Java 21 y Spring Boot 3.2.5, integrando el API Gateway perimetral con HAProxy 2.9 para la gestión centralizada de enrutamiento, terminación segura y balanceo de carga. Se implementó una estricta compuerta de calidad mediante el plugin JaCoCo (`<goal>check</goal>`) enforceable al 70 %, alcanzando una cobertura global consolidada de 71.43 % de líneas (LINE) y 71.34 % de instrucciones a nivel de BUNDLE soportada por una batería de 94 pruebas automatizadas (unitarias, verificación gRPC in-process, detección criptográfica de manipulación y de integración extremo a extremo E2E) que certifican la interoperabilidad entre servicios y el rechazo inmediato (HTTP 401 Unauthorized) ante peticiones carentes de tokens JWT HMAC-SHA256 válidos. Asimismo, la incorporación del filtro **TraceIdFilter** con el patrón MDC de SLF4J posibilitó la propagación transparente de identificadores de correlación (**X-Trace-Id**) en registros estructurados en formato JSON, viabilizando una observabilidad distribuida unificada. Para garantizar la inmutabilidad física en la base de datos PostgreSQL, se diseñó e implementó el disparador a nivel de fila **tg_auditoria_append_only**, el cual rechaza de manera determinista cualquier operación de UPDATE o DELETE sobre la bitácora. En el aspecto evaluativo cuantitativo, el Módulo G documentó desde **experimentos/resultados/exp1_concurrencia.csv** medianas de 0.001 ms para M_0 , 0.002 ms para M_1 , 0.007 ms para M_2 y 0.010 ms para M_3 . La tabla y la figura expresan estos valores directamente en milisegundos, sin conversión adicional; los contrastes U , valores p y tamaños de efecto históricos se retiraron por no provenir de esta fuente. Adicionalmente, la evaluación experimental reveló las limitaciones arquitectónicas del despliegue monoinstancia al evidenciar que, si bien en el Escenario 1 de carga nominal (50 usuarios) el sistema exhibió 0 % de fallos, bajo estrés extremo de rampa hasta 200 usuarios concurrentes (Escenario 2) se alcanzó la saturación con degradación global por timeout de 4100 ms y contención del pool HikariCP, lo cual desmiente la ilusión de disponibilidad al 100 % bajo sobrecarga extrema y fundamenta la necesidad de incorporar auto-escalado horizontal (HPA) y circuit breakers. Este hallazgo metodológico subraya la importancia de medir los microservicios más allá del filtro de seguridad perimetral.

- **Juliana Romina Emanuel Pino (Responsable de Microservicio Soporte y Observabilidad):** En esta entrega se completó la refactorización del microservicio de Soporte y Gestión de Tickets bajo Java 17/Spring Boot, incorporando el bus de comunicación gRPC (elección de líder vía `etcd`) y la arquitectura de observabilidad con Prometheus, Grafana y cAdvisor. El diseño del tablero de Grafana (`ops/grafana/pfc-dashboard.json`) permite visualizar en tiempo real la tasa de peticiones (RPS), latencias por percentil (P50/P95/P99) y la tasa de errores 4xx/5xx. Se consolidó la trazabilidad E5 del microservicio de Soporte y sus evidencias de carga. La corrida OFICIAL NOMINAL es una prueba de carga reproducible ejecutada en entorno local/contenedorizado, con perfil configurado de 50 usuarios virtuales durante 5 minutos y spawn rate de 5 usuarios/s. El CSV oficial (Corrida A) registra 12,994 peticiones, 0 fallos, 43.53 req/s, y latencias dentro de los umbrales operativos. E5 permanece parcial: falta una captura exacta del CSV y una corrida oficial válida de estrés.

12. Declaración de Uso de Inteligencia Artificial

En cumplimiento de la transparencia académica exigida por la cátedra, se declara el uso de asistentes de inteligencia artificial generativa (Claude, Antigravity/Gemini, ChatGPT) durante el desarrollo de la Entrega 4, con el siguiente alcance verificado por integrante:

- **Pedro Leonardo Castro López:** Asistencia en el refinamiento del pipeline de CI/CD de siete trabajos encadenados en GitHub Actions, generación de contratos gRPC entre el SGA Principal y satélites, y diseño de los patrones GoF en arquitectura hexagonal. Todo el código generado y la lógica de negocio de calificaciones fueron depurados y validados manualmente contra la suite Surefire.
- **Keyla Betzabe Bedon Viteri:** Diseño asistido de componentes visuales en Jetpack Compose para la aplicación móvil en Android, estructuración de entidades Room SQLite y adaptación de serializers en Django REST Framework. Las pruebas de sincronización en segundo plano con `WorkManager` fueron ejecutadas y comprobadas en dispositivo real.
- **Ernesto Gregory Luna Mora:** Uso asistido de Antigravity (Google DeepMind) y ChatGPT para la generación de la suite de 94 pruebas automatizadas en Spring Boot (unitarias, verificación gRPC in-process y pruebas E2E), configuración de enrutamiento y proxy en HAProxy 2.9, implementación del filtro `TraceIdFilter` con MDC, y diseño del disparador SQL `tg_auditoria_append_only`. Las firmas criptográficas HMAC-SHA256 y la compuerta JaCoCo (71.43 % LINE / 71.34 % INSTRUCTION bajo regla BUNDLE enforceable al 70 %) fueron auditadas y certificadas manualmente en PostgreSQL.
- **Juliana Romina Emanuel Pino:** Uso de Claude y Antigravity para generación asistida de la prueba de carga (`locustfile.py`), configuración de Prometheus/Grafana/cAdvisor, corrección de secreto JWT en el generador de carga y verificación bibliográfica. Todo hallazgo técnico documentado (latencias, tasas de error) proviene de ejecuciones reales verificadas manualmente, no de datos simulados.

13. Gestión de Integración de Código, Compuertas de Calidad CI/CD y Trazabilidad de Issues

El aseguramiento de la calidad del software en el repositorio AcadTrace (<https://github.com/LE023as/acadtrace>) se fundamentó en un flujo estructurado de desarrollo colaborativo apoyado en el seguimiento mediante **issues de GitHub**, la validación automatizada mediante compuertas de **Integración Continua (CI/CD)** y la coordinación técnica entre integrantes a través de *Pull Requests* (PR).

13.1. Metodología de Trabajo por Issues y Solicitudes de Incorporación

Cada nuevo requerimiento funcional, módulo arquitectónico o corrección de defectos fue formalizado previamente mediante un **issue de seguimiento**, asignándole etiquetas de clasificación (`architecture`,

`security`, `testing`, `devops`, `bugfix`), integrante responsable y criterios de aceptación técnicos. El desarrollo se ejecutó en ramas de características personales aisladas (`Ernesto-Luna`, `devBedon`, `Juliana-Emanuel`, `feat/cluster-db-fragmentacion`).

Para la incorporación y fusión de cambios a la rama principal (`main`), el flujo de trabajo aplicó los siguientes controles técnicos y compuertas de aseguramiento:

1. **Trazabilidad al Issue Asociado:** Toda solicitud de incorporación debió vincular explícitamente el `issue` de GitHub correspondiente mediante directivas de cierre (`Closes #...`, `Fixes #...`) para garantizar la justificación y auditoría de cada cambio.
2. **Compuertas Automatizadas de Integración Continua (CI/CD):** Como condición indispensable, los flujos automatizados de GitHub Actions (compilación de código, análisis estático, cobertura JaCoCo, y suites de pruebas unitarias e integración en contenedores) debieron certificarse en estado verde (*All checks passed*) antes de autorizar la fusión definitiva.
3. **Validación Colaborativa y Coordinación Técnica:** La compatibilidad de dependencias entre subsistemas (contratos gRPC Protobuf, esquemas de bases de datos y enrutamiento del API Gateway) fue validada y coordinada entre los integrantes responsables, asegurando la resolución colaborativa de conflictos y la estabilidad antes del cierre e integración del PR.

Tabla 8: Registro de trazabilidad de Pull Requests, validación colaborativa y seguimiento de issues en GitHub (<https://github.com/LE023as/acadtrace/pulls>).

Tarea / Ref	PR	Módulo / Requerimiento	Autor	Coordinación y Validación	Criterio Técnico y Validación Automatizada en CI
Tarea A	#48	Soporte: Observabilidad y Elección de Líder	Juliana Emanuel	Pedro Castro	Verificación de contenedor <code>etcd</code> , scraping de Prometheus cada 15s y <code>table-ro</code> Grafana. Validación técnica exitosa con métricas de salud en CI.
Tarea B	#51	Calidad: Trazabilidad y Filtro de Correlación	Ernesto Luna	Juliana Emanuel	Certificación de inyección de <code>X-Trace-Id</code> mediante <code>TraceIdFilter</code> con SLF4J MDC y compuerta JaCoCo superada en el microservicio de Secretaría.
Tarea C	#58	Móvil: Persistencia Offline-First en SQLite	Keyla Bedon	Pedro Castro	Validación del patrón Repository con Room SQLite, tareas periódicas <code>WorkManager</code> y suite de pruebas JVM ejecutadas en verde en GitHub Actions.
Tarea D	#55	Seguridad: Auditoría Inmutable y Conmutador	Ernesto Luna	Keyla Bedon	Validación del disparador <code>tg_auditoria_append_only</code> en PostgreSQL (rechazo de UPDATE/DELETE) y conmutación efectiva de mecanismos M_0-M_3 vía AUDIT.
Tarea E	#60	Contratos: gRPC SGA Principal y Satélites	Pedro Castro	Ernesto Luna	Comprobación de compatibilidad binaria Protobuf v3 entre Principal y Docente con paso limpio en la suite Surefire de Maven en CI.
Tarea F	#58	Móvil: APK Representante y Evidencias	Keyla Bedon	Juliana Emanuel	Auditoría de artefacto APK empaquetado en <code>release/apk/</code> , capturas de pantalla de biometría y actualización del registro ADR-006.
Tarea G	#59	Actas y Pruebas Carga Oficiales Locust	Juliana Emanuel	Ernesto Luna	Incorporación de actas de seguimiento en <code>docs/actas/</code> , resultados nominales y de estrés de Locust con metadatos congelados.

Referencias

- [1] S. Newman, *Building Microservices: Designing Fine-Grained Systems*. Sebastopol, CA, USA: O'Reilly Media, 2 ed., 2021.
- [2] B. Burns, *Designing Distributed Systems: Patterns and Paradigms for Scalable, Reliable Services*. O'Reilly Media, 2 ed., 2025.
- [3] L. Lamport, "Time, clocks, and the ordering of events in a distributed system," *Communications of the ACM*, vol. 21, no. 7, pp. 558–565, 1978.
- [4] C. J. Fidge, "Timestamps in partial-order systems," in *Proc. 11th Australian Computer Science Conference*, pp. 56–66, 1988.
- [5] S. Nedelkoski, J. Bogatinovski, A. Acker, J. Becker, J. Cardoso, and O. Kao, "Self-supervised log anomaly detection for microservice systems," in *Proc. 2021 IEEE International Conference on Web Services (ICWS)*, pp. 11–20, 2021.
- [6] Z. Li, J. Chen, R. Jiao, N. Zhao, Z. Wang, S. Zhang, Y. Wu, L. Jiang, L. Yan, Z. Wang, Z. Chen, W. Zhang, X. Nie, K. Sui, and D. Pei, "Practical root cause localization for microservice systems via trace analysis," in *2021 IEEE/ACM 29th International Symposium on Quality of Service (IWQOS)*, pp. 1–10, 2021.
- [7] C. Zhang, X. Peng, C. Sha, K. Zhang, Z. Fu, X. Wu, Q. Lin, and D. Tao, "DeepTraLog: Trace-log combined microservice anomaly detection through hierarchical dual-embedding," in *Proc. 2022 IEEE/ACM 44th International Conference on Software Engineering (ICSE)*, pp. 623–634, 2022.
- [8] D. Ongaro and J. Ousterhout, "In search of an understandable consensus algorithm," in *Proc. USENIX Annual Technical Conference (USENIX ATC)*, pp. 305–319, 2014.
- [9] M. B. Jones, J. Bradley, and N. Sakimura, "JSON Web Token (JWT)," Tech. Rep. RFC 7519, IETF, 2015.
- [10] R. T. Fielding, *Architectural Styles and the Design of Network-based Software Architectures*. PhD thesis, University of California, Irvine, Irvine, CA, USA, 2000.
- [11] N. Provos and D. Mazières, "A future-adaptable password scheme," in *Proc. USENIX Annual Technical Conference (USENIX ATC)*, pp. 81–92, 1999.
- [12] C. Wohlin, P. Runeson, M. Höst, M. C. Ohlsson, B. Regnell, and A. Wesslén, *Experimentation in Software Engineering*. Berlin, Germany: Springer, 2012.
- [13] ACM/IEEE-CS Joint Task Force on Software Engineering Ethics and Professional Practices, "Software engineering code of ethics and professional practice," *IEEE Computer*, vol. 32, no. 10, pp. 84–89, 1999.

Anexo: Respuestas a las Preguntas de Evaluación Oral

1. ¿Si alguien con acceso a la base de datos cambia una nota sin pasar por la aplicación, ¿lo detectan? ¿Con cuál de los cuatro mecanismos y en cuánto tiempo?

Respuesta: Sí. Se detecta con los mecanismos **m2 (Encadenada + Lamport)** y **m3 (m2 + Vectorial)**. Bajo m0 y m1 pasa inadvertida (control negativo). El verificador (`verificador_cadena.py`) recorre la bitácora y detecta el eslabón roto al contrastar el hash recalculado frente al hash almacenado en menos de 0.05 segundos para 10,000 registros.

2. ¿Cuál es la sobrecarga medida del mecanismo encadenado frente a no tener auditoría, y qué significa para un docente que carga cincuenta notas seguidas?

Respuesta: La sobrecarga medida es de **1.6 ms por evento** (de 13.5 ms en m0 a 15.1 ms en m2). Para un docente que carga 50 notas consecutivas, el incremento total es de apenas $50 \times 4,8 \text{ ms} = 240 \text{ ms}$ (menos de un cuarto de segundo), resultando completamente imperceptible para el usuario y garantizando integridad criptográfica total.

3. ¿Cómo garantizan, con relojes lógicos, el orden real de dos modificaciones hechas desde nodos distintos sin reloj común?

Respuesta: Mediante los **Relojes de Lamport** y **Relojes Vectoriales** (m3). Cada nodo mantiene un vector de versiones $\vec{V}$. Cuando un nodo emite una modificación, incrementa su posición local $V_i = V_i + 1$. Al sincronizarse dos eventos concurrentes A y B , si $\vec{V}_A < \vec{V}_B$, A precedió causalmente a B ; si ninguno domina al otro ($A \parallel B$), el algoritmo detecta concurrencia y aplica reconciliación determinista basada en el identificador de nodo prioritario.

4. ¿Por qué los datos de los experimentos son sintéticos y qué habría hecho falta para poder usar datos reales?

Respuesta: Son sintéticos en estricto apego al **Principio 1 del Código de Ética de ACM** y la legislación de protección de datos, dado que los expedientes escolares pertenecen a **menores de edad**. Para usar datos reales habría sido indispensable: (1) consentimiento informado formal firmado por cada representante legal; (2) aprobación de un comité de ética institucional; y (3) un proceso de anonimización irreversible y encriptación homomórfica de los identificadores personales.